

Lab Practical 05

Lab Practical Number	Lab 05 — “Hello, Agent”
Related Lecture	Lecture 05 – Agentic Software Development 1
Duration	2 hours in-lab + approx. 30 minutes compulsory pre-lab setup (see page 3)
Learning Outcomes	LO1 • LO2 • LO3 • LO4
Cost to student	Rs. 0 — Google AI Studio (Gemini API) free tier. No credit card, no cloud billing.
Lab pack	SE3090_Lab05_Agentic_AI_Part_1.zip (download from CourseWeb before the lab)
Mode	Individual work • Word document + evidence file submitted to CourseWeb
Marks	20 marks (contributes to the Lab Submission component)

Introduction

Lecture 05 introduced agentic software development: what an AI agent actually is, how it differs from a plain chatbot, and where agents fit into a modern engineering workflow. In this lab you stop reading about agents and build one — four times over, at four increasing levels of abstraction — so that you can say precisely what each layer of a framework is doing for you and what it is hiding.

THE ONE IDEA BEHIND THE WHOLE LAB

An agent is a language model in a loop with tools and state.

The model cannot run anything. It can only produce text — including a specially formatted request asking your program to run a function. The loop, the execution and the safety limits are all your code. Once you have written those fifteen lines by hand, no agent framework can confuse you again: you will simply be looking for where it keeps its version of your loop.

Continuity: Lab 02 built the React frontend and Lab 04 built the secured ASP.NET Core API and PostgreSQL database. This lab builds the third piece — the agent — and ships it behind an HTTP endpoint so that a frontend can call it exactly like any other REST service. That is the same integration pattern you will need for the agentic AI feature in your Main Assignment.

What you will have built by the end:

```
raw chat call      -> text in, text out, no memory of anything
|
tool-calling by hand -> the model asks; YOUR code executes;
|                       repeat until it stops asking
create_agent      -> the same loop, with retries / streaming /
|                       caps supplied by a framework
FastAPI service    -> the loop behind an HTTP endpoint, returning
|                       the answer AND its trace
```

The same capability, four times, at four altitudes.

Lab Objectives

By the end of this lab, you should be able to:

- **Explain why the chat API is stateless** and demonstrate where “memory” actually lives in an AI application (LO1).

- **Read a tool's derived JSON schema** and state exactly which part of your Python function produced each part of it (LO1, LO2).
- **Write the agent loop from memory** — invoke, check tool_calls, execute, append ToolMessage, repeat (LO2).
- **Name at least three agent failure modes** and implement the code-level guardrail for each (LO3).
- **Expose an agent over HTTP** with request validation, an iteration cap and a machine-readable trace (LO2, LO3).
- **Justify the level of abstraction you choose** — hand-rolled loop versus framework — for a given engineering context (LO4).

Required Tools and Materials

Item	Details
Python	Version 3.11 or newer. Check with: <code>python3 --version</code>
Lab pack	SE3090_Lab05_Agentic_AI_Part_1.zip — contains lab.ipynb, solutions.ipynb, demos.ipynb, api/main.py, requirements.txt, .env.example
Editor	JupyterLab (installed by requirements.txt) or VS Code with the Jupyter extension
API credential	One free Google AI Studio API key, created by you, on your own Google account
Network	Access to <code>generativelanguage.googleapis.com</code> (campus Wi-Fi is fine; some VPNs block it)
Reference	Lecture 05 slide deck — the definitions and the ReAct diagram are examinable

NO CREDIT CARD, NO CLOUD ACCOUNT, NO COST

Everything in this lab runs on the Google AI Studio free tier. You will not be asked for a payment method at any point. If a page asks you for billing details, you are on the wrong page — stop and ask a demonstrator.

BEFORE THE LAB • APPROX. 30 MINUTES • COMPULSORY

Pre-Lab Setup — Do This At Home

READ THIS FIRST

Do not arrive at the lab with an empty folder. Installing the packages takes 10–25 minutes and forty students downloading them at once will saturate the lab network. Complete Steps 1–6 below before the session and arrive with a green kernel check. Students who complete setup at home reliably finish the whole lab; students who start at 0:00 rarely get past Task 03.

Step 1 — Check your Python version

```
python3 --version
```

Expected: Python 3.11.x or newer. On Windows, try 'python --version' instead.

If it is older than 3.11: Windows — install from python.org and tick “Add python.exe to PATH”. macOS — brew install python@3.12. Linux — use your package manager.

Step 2 — Unzip the lab pack and open a terminal inside it

```
cd path/to/SE3090_Lab05_Agentic_AI_Part_1/lab5
ls
```

Expected: you see lab.ipynb, requirements.txt, api, .env.example

Step 3 — Create the virtual environment and install

```
python3 -m venv .venv
```

```
# macOS / Linux:
```

```
source .venv/bin/activate
```

```
# Windows PowerShell:
```

```
.venv\Scripts\Activate.ps1
```

```
pip install -U pip
```

```
pip install -r requirements.txt
```

Expected: the last line ends with 'Successfully installed ...' and your prompt now starts with (.venv).

Verify: `python -c "import langchain, fastapi; print('ok')"` should print ok.

Step 4 — Register the Jupyter kernel

```
python -m ipykernel install --user --name agentic-w5 \
    --display-name "Agentic AI (week 5)"
```

Expected: 'Installed kernelspec agentic-w5 in ...'. On Windows, put it all on one line without the backslash.

Verify: `jupyter kernelspec list` must contain agentic-w5.

Step 5 — Get your own free Google AI Studio API key

1. Open <https://aistudio.google.com/apikey> and sign in with a personal Google account.
2. Click Create API key. Accept the default project if you are prompted.
3. Copy the key. It begins with AIzaSy and has no spaces.

YOUR KEY IS A PASSWORD — AND YOUR QUOTA IS PER PROJECT, NOT PER KEY

Every student must create their own key on their own Google account. Google applies rate limits per Google Cloud project, not per API key — so making three keys inside one project gives you no extra quota, and sharing one key across a lab bench pools the whole bench into one person’s limit. That is the fastest way to spend the session staring at 429 errors.

Never commit .env to Git. The repository .gitignore already excludes it. A leaked key can be used by anyone until you revoke it.

Step 6 — Create your .env file

```
# macOS / Linux:
cp .env.example .env
# Windows:
copy .env.example .env
```

Then open .env in any text editor and edit the two lines below.

```
GOOGLE_API_KEY=AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
CHAT_MODEL=gemini-3.5-flash-lite
```

.env after editing. Paste your real key; keep the model ID exactly as shown.

Model IDs are plain names. There is no publisher prefix and no endpoint URL — the library already knows where the Gemini API lives. `gemini-3.5-flash-lite` is the course default because it has the most generous free-tier request rate and handles every cell in this lab. If you want stronger reasoning and are not near your limit, you may switch to `gemini-2.5-flash`. Newer free-tier Flash-Lite models work too — the model ID is the only thing that changes.

Step 7 — Launch and confirm the kernel

```
jupyter lab lab.ipynb
```

Then check the kernel indicator at the TOP-RIGHT of the notebook.

It must read **“Agentic AI (week 5)”**. If it says “Python 3”, change it: **Kernel ▶ Change Kernel... ▶ Agentic AI (week 5)**.

WHY “BUT I INSTALLED EVERYTHING” DOES NOT HELP

Activating `.venv` changes that terminal. It does not change which Python JupyterLab hands your cells to — that is the kernel, a separate process. On the default “Python 3” kernel your cells run on system Python, where none of the lab packages exist. So you get `ModuleNotFoundError` even though `pip list` in your terminal looks perfect. The two facts are not in conflict; they are about two different Pythons.

The trick that avoids this whole class of problem: launch Jupyter from inside the activated `venv` — `python -m jupyterlab lab.ipynb` — so even the default kernel is the right interpreter.

Pre-lab setup troubleshooting

Exact error text you see	What it means	Fix
<code>ModuleNotFoundError: No module named 'langchain'</code>	Wrong Jupyter kernel selected	Kernel ▶ Change Kernel... ▶ Agentic AI (week 5). If it is not listed, redo Step 4.
<code>'python3'</code> is not recognized as an internal or external command	Windows uses a different name	Use <code>python</code> instead of <code>python3</code> in every command.
cannot be loaded because running scripts is disabled on this system	PowerShell execution policy blocks <code>venv</code> activation	Run: <code>Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass</code> then activate again.
error: externally-managed-environment	You are installing outside the <code>venv</code>	Activate <code>.venv</code> first (Step 3). The prompt must show <code>(.venv)</code> .
No kernel named <code>agentic-w5</code>	Kernel never registered	Activate <code>.venv</code> , rerun Step 4, then reload the browser tab.
Nothing works and the lab starts in 5 minutes	—	In a notebook cell run <code>%pip install -r requirements.txt</code> then restart the kernel. <code>%pip</code> installs into whichever kernel you are actually on.

READ BEFORE YOU RUN ANYTHING

Working Within the Free Tier

This lab is designed to fit comfortably inside the Google AI Studio free tier, but only if you pace yourself. Google measures your usage in three dimensions at once, and exceeding any single one returns the same error.

Dimension	What it counts	Which one will bite you in this lab
RPM	Requests per minute	This one. The lab fires bursts of 3–6 requests within a few seconds.
TPM	Input tokens per minute	Very unlikely — the prompts here are tiny.
RPD	Requests per day	Unlikely. A full pass plus reruns stays well inside a normal day's allowance.

Google no longer publishes a fixed free-tier table in its documentation; your live limits are shown for your own project at <https://aistudio.google.com/rate-limit>. Check yours before the lab. Daily quotas reset at midnight US Pacific time.

Your request budget for this lab

Task	Requests	Note
Task 00 — environment check	1	One call, purely to confirm the key works.
Task 01 — raw chat	9	1 + 2 (amnesia) + 2 (history) + 4 (temperature).
Task 02 — the loop by hand	7	1 to inspect a tool call, then 3 per run of the agent.
Task 03 — create_agent	6	One invoke run and one streamed run.
Task 04 — your own agent	8	Your agent, plus the two break-it experiments.
Task 05 — the API endpoint	9	Three endpoint calls at roughly 3 model calls each.
Clean single pass	≈ 40	Allow 60–90 in practice — you will rerun cells while debugging.

FOUR RULES THAT KEEP YOU UNDER THE LIMIT

- 1. Do not machine-gun the cells.** Read the output of each cell before running the next. This is not politeness — it is rate-limit management, and it is also how you learn something.
- 2. A 429 means “wait”, not “stop”.** ResourceExhausted almost always means you exceeded requests-per-minute. Wait 60 seconds and rerun the same cell.
- 3. If 429s keep coming, switch models.** Set `CHAT_MODEL=gemini-2.5-flash-lite` in `.env` and restart the kernel. It runs every cell in this lab.
- 4. Never share a key.** Limits are per project. One shared key means one shared quota for the whole bench.

Every model client in this lab is created with `max_retries=3` and `timeout=60`. The retries absorb short bursts automatically by backing off and trying again — but retrying cannot create quota that you do not have. Pacing is still your job.

Lab Time Plan (120 minutes)

Time	Activity	You produce
0:00 – 0:10	Task 00 — environment check and briefing	The model prints OK
0:10 – 0:30	Task 01 — raw chat: statelessness, history, temperature	Checkpoint 1
0:30 – 1:05	Task 02 — the hand-rolled agent loop (core of the lab)	Checkpoint 2
1:05 – 1:20	Task 03 — create_agent: the same loop, industrialised	Checkpoint 3
1:20 – 1:40	Task 04 — your own agent, then break it	Checkpoint 4

Time	Activity	You produce
1:40 – 1:55	Task 05 — ship the agent behind a FastAPI endpoint	Checkpoint 5
1:55 – 2:00	Quick test, evidence file, submit to CourseWeb	Submitted files

HOW TO USE THIS PLAN

Task 02 carries the most marks and is the whole point of the lab. If you fall behind, protect Task 02 and Task 05 — Task 04 can be shortened to a single break-it experiment. Do not skip the quick test; it is 4 of the 20 marks.

TASK 00 • 10 MINUTES • LO2

Environment Check

Open lab.ipynb. Run the first three cells in order. Nothing here teaches agents — it proves your environment is real before you depend on it.

Step-by-step instructions

1. Run the **kernel check** cell. It imports only the standard library, so it works even when nothing else is installed. It prints which interpreter you are on and which packages are missing.
2. Run the configuration cell. It loads .env and asserts your key is present.
3. Run the model cell. This is your first and only request in this task.

What the configuration cell does, line by line

```
import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv(Path.cwd() / ".env")      # THIS folder's .env, not a shared one

API_KEY    = os.getenv("GOOGLE_API_KEY", "")
CHAT_MODEL = os.getenv("CHAT_MODEL", "gemini-2.5-flash-lite")

assert API_KEY and "XXXX" not in API_KEY, (
    "GOOGLE_API_KEY missing: copy .env.example -> .env and paste "
    "your Google AI Studio key"
)
```

- `load_dotenv(Path.cwd() / ".env")` is explicit about the path, so the notebook behaves identically whether you launched Jupyter from this folder or from somewhere else.
- `os.getenv(name, default)` supplies a working default, so a half-filled .env still runs.
- The `assert` fails loudly and early, with the fix in the message. A missing credential should never first appear as a confusing HTTP error forty lines later.

The model client

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model=CHAT_MODEL,
    google_api_key=API_KEY,
    temperature=0,      # near-deterministic: debug behaviour, not variety
    timeout=60,         # a hung request must fail, not wedge the notebook
    max_retries=3,      # back off and retry on a transient 429
)

print(llm.invoke("Reply with exactly: OK").content)
```

EXPECTED OUTPUT

The single word **OK**. That means your key, your model ID and your network are all correct. It is your green light for the rest of the lab.

TASK 01 • 20 MINUTES • LO1

Raw Chat: Statelessness, Memory and Temperature

Before you can reason about agents you need two facts in your fingers: the chat API has no memory, and its output is sampled rather than computed. Almost every confusing agent bug you will ever meet is one of these two facts wearing a disguise.

The four message types

Class	Represents	Used for
SystemMessage	The developer	Persona, rules, tool-use policy. Re-sent on every single call.
HumanMessage	The user	The question or the instruction.
AIMessage	The model	Prose, or a request to call tools (.tool_calls).
ToolMessage	Your runtime	The result of a tool that you executed, linked back by tool_call_id.

Step 1 — One call, and a useful habit

```
from langchain_core.messages import (
    AIMessage, HumanMessage, SystemMessage,
)

def text_of(msg):
    """Return an AIMessage's text whether .content is a string or a
    list of content blocks. Gemini can return either."""
    c = msg.content
    if isinstance(c, list):
        return "".join(b.get("text", "") for b in c
                        if isinstance(b, dict))
    return c

response = llm.invoke([
    HumanMessage("In one sentence, what is an AI agent?")
])
print("reply :", text_of(response))
print("tokens:", response.usage_metadata)
```

WHY text_of() EXISTS — AND WHY YOU SHOULD KEEP USING IT

Gemini may return `.content` as a plain string, or as a list of content blocks. Code that assumes a string will crash with `AttributeError: 'list' object has no attribute 'strip'` the first time it meets the other shape. Use `text_of()` everywhere in this lab instead of touching `.content` directly.

`usage_metadata` is LangChain's provider-neutral token count: `input_tokens`, `output_tokens`, `total_tokens` — the same three keys whichever model you point at. Look at it now, because in Task 02 you will watch `input_tokens` climb on every iteration. That number is the price of memory.

Step 2 — The amnesia demonstration

```
llm.invoke([HumanMessage("Hi! My name is Nadia.")])

response = llm.invoke([HumanMessage("What is my name?")])
print(text_of(response))      # it has no idea
```

Two separate HTTP requests. The first response is discarded; the second request contains exactly one message. The model is not being forgetful — nothing was ever stored. The endpoint is a pure function of the messages you send it.

Step 3 — The fix: memory is a Python list that you re-send

```
history = [
    SystemMessage("You are a concise, helpful assistant."),
    HumanMessage("Hi! My name is Nadia."),
]
reply = llm.invoke(history)
history.append(reply)          # keep the assistant turn
history.append(HumanMessage("What is my name?"))

print(text_of(llm.invoke(history)))    # now it 'remembers'
```

The whole of “memory” is this: a list that you re-send. Every memory feature you will ever meet — checkpoints, summarisers, vector memory, “threads” — is a strategy for deciding what goes into this list and what gets dropped from it. Lecture 06 makes that machinery explicit. Today it is four lines you can see.

Step 4 — Temperature (4 requests — pace yourself)

```
prompt = [HumanMessage(
    "Give a name for a coffee shop run by robots. Name only."
)]

for temp in (0.0, 1.2):
    model = ChatGoogleGenerativeAI(
        model=CHAT_MODEL, google_api_key=API_KEY,
        temperature=temp, timeout=60, max_retries=3,
    )
    names = [text_of(model.invoke(prompt)).strip()
              for _ in range(2)]
    print(f"temperature={temp}: {names}")
```

Two samples per temperature, not three: same lesson, one third fewer requests.

At 0.0 the samples are near-identical; at 1.2 they diverge. Temperature scales the randomness of token sampling. The consequence for this lab is important: your agent is not a function. Two runs of identical code can take different paths — which is why evaluating agents needs more than an assert statement.

Your turn — Exercise 1.1

Change the SystemMessage in Step 3 to a persona of your choice — a pirate, a strict librarian, a terse site-reliability engineer. Rerun it. Then **delete that message from history** and rerun again, and watch the persona vanish. Confirm for yourself that the persona holds only because that message is re-sent on every call.

CHECKPOINT 1 — SAVE THIS OUTPUT

Capture the amnesia demonstration, the fixed version, and your persona variant.

A demonstrator may ask you: *where does the conversation actually live?*

TASK 02 • 35 MINUTES • LO2, LO3 • CORE OF THE LAB

The Hand-Rolled Agent Loop

TYPE THIS ONE YOURSELF

Reference implementations are provided in solutions.ipynb. Open them after an honest attempt, not before. Copying this section defeats the entire purpose of the lab — and this is the section the final exam draws from.

Frameworks make agents look magical. They are not. The whole mechanism is: the model emits a structured request; your code executes it; you append the result and ask again. Fifteen lines.

Step 1 — Defining tools

```
import re
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Current weather for a city. Use whenever the user asks
    about weather."""
    fake = {
        "colombo": "31 C, humid, thunderstorms",
        "kandy": "24 C, light rain",
        "jaffna": "33 C, clear",
    }
    return fake.get(city.lower(), f"No data for {city}")

@tool
def calculator(expression: str) -> str:
    """Evaluate an arithmetic expression, e.g. '23*4+1'.
    Digits and + - * / ( ) only."""
    if not re.fullmatch(r"[0-9+\-*/()\. ]+", expression):
        return "Error: invalid characters"
    return str(eval(expression)) # safe ONLY due to the whitelist
```

What the `@tool` decorator does, precisely:

- reads the **function name** → becomes the tool name the model must emit;
- reads the **docstring** → becomes the description the model reads when deciding;
- reads the **type hints** → become a JSON schema for the arguments;
- wraps the function so it can be called with a dict: `get_weather.invoke({"city": "Colombo"})`.

TWO DETAILS WORTH INTERNALISING

1. The docstring is not a comment — it is the interface. It is the only documentation the model has. “Does stuff” is a bug, and you will prove that to yourself in Task 04.

2. `eval()` is acceptable here only because of the whitelist. The regex admits digits, operators, brackets, dots and spaces — no letters, therefore no names, no attribute access, no function calls. Notice where the safety lives: in your Python, not in a polite request to the model.

Do not copy `eval()` into your Main Assignment. Even whitelisted, it is a pattern reviewers flag. In production use a proper expression parser such as `ast.literal_eval` or a small arithmetic library.

Step 2 — See exactly what the model sees

```
import json
print(json.dumps(
    get_weather.args_schema.model_json_schema(), indent=2
))
print("description:", get_weather.description)
```

```
{
  "description": "Current weather for a city. Use whenever ...",
  "properties": {"city": {"title": "City", "type": "string"}},
  "required": ["city"],
  "title": "get_weather",
  "type": "object"
}
```

Every part of that JSON came from your function signature and docstring. Nothing else about your code is visible to the model.

Your turn — TODO 1: bind the tools and inspect one raw tool call

Specification. Bind the two tools to the model, invoke it with a question that clearly needs one, then print both `response.content` and `response.tool_calls`.

```
content    : ''
tool_calls: [{'name': 'get_weather',
               'args': {'city': 'Colombo'},
               'id': 'call_abc123',
               'type': 'tool_call'}]
```

Expected shape of the output.

Stop and read that. `content` is empty — the model produced no prose because it decided the next useful act was a function call.

Key	Meaning
name	Which tool it wants. It can hallucinate a name that does not exist — your code must cope.
args	Arguments, already parsed into a dict against your schema.
id	A correlation ID. The matching ToolMessage must carry it, or the provider cannot pair result to request.

`bind_tools` does not change the model. It attaches the tool schemas to every request made through the object it returns.

Your turn — TODO 2: write the loop

Specification. Write `run_agent(user_input, max_iterations=5):`

1. Start messages with the system prompt and the user's question.
2. Loop at most `max_iterations` times. Each iteration: (a) invoke `llm_with_tools` with messages and append the response; (b) if the response has no `tool_calls`, it is the final answer — return its text; (c) otherwise, for each call: print a trace line, execute the tool, and append a `ToolMessage` carrying the result and the call's id.
3. If the loop finishes without an answer, return a "stopped" message.

The parts that matter, and why they are written that way

Line	Why
<code>messages = [SystemMessage(...), HumanMessage(...)]</code>	The transcript is a plain list. Nothing is hidden inside an object you cannot print.
<code>for step in range(max_iterations)</code>	The termination guarantee. Without it a confused model can loop until your quota is gone.
<code>response = llm_with_tools.invoke(messages)</code>	The entire transcript is re-sent every iteration. This is why cost grows faster than steps.
<code>messages.append(response)</code>	The model's own request must stay in the transcript, or its follow-up call will make no sense.
<code>if not response.tool_calls: return ...</code>	The exit condition. Prose means done — the model signals completion by choosing to speak.
<code>TOOLS.get(call["name"])</code> then a <code>None</code> check	Failure mode 1: a hallucinated tool name. Returning an error string lets the model correct itself; <code>TOOLS[...]</code> would raise <code>KeyError</code> and kill the run.
<code>try / except</code> around <code>.invoke</code>	Failure mode 2: the tool itself fails. Turn failures into observations — the model can act on an observation, but never on a traceback.
<code>ToolMessage(content=..., tool_call_id=call["id"])</code>	The result must be tied back to the request by ID.
<code>return "Stopped: ..." after the loop</code>	Failure mode 3: runaway loops. The guardrail is an integer in your code, not a sentence in a prompt.

Step 3 — Run it on a task that needs two tools

```
question = (
    "If it's above 30 C in Colombo I need 3 fans at Rs. 4500 "
    "each, otherwise 1. What do I spend?"
)
print("Answer:", run_agent(question))
```

```
[step 0] get_weather({'city': 'Colombo'})
[step 1] calculator({'expression': '3*4500'})
Answer: It's 31 C in Colombo, so you need 3 fans: Rs. 13,500.
```

Expected trace. Yours may differ slightly — that is non-determinism, not a bug.

Read what happened. The model called one tool, read the result, and chose the second call *because of* that result. Nothing in your code encodes “check the weather, then multiply”. That is **ReAct** — reason, act, observe, repeat — and you have just written its runtime.

Your turn — Exercise 2.4: instrumentation

Extend your loop so it also reports the number of messages in the final transcript and the total tokens consumed. Sum `response.usage_metadata["total_tokens"]` across the iterations. Then answer in a comment: **why do the tokens grow faster than the number of steps?**

CHECKPOINT 2 — SAVE THIS OUTPUT

Capture your trace for the fans question, plus the message count and token count.

A demonstrator may ask you: *point to the exact line where “the runtime disposes”.*

TASK 03 • 15 MINUTES • LO1, LO4

create_agent — The Same Loop, Industrialised

```

from langchain.agents import create_agent

agent = create_agent(
    llm,
    tools=[get_weather, calculator],
    system_prompt=SYSTEM_PROMPT,
)
result = agent.invoke({
    "messages": [{"role": "user", "content": question}]
})
print(result["messages"][-1].content)

```

Three lines replace your Task 02 loop. Note the interface differences — they are not cosmetic:

- the input is a **dict with a messages key**, not a bare list, because underneath this is a LangGraph state machine (Lecture 06's topic) and state machines take states;
- messages may be plain dicts (`{"role": "user", ...}`) as well as message objects;
- the return value is the **whole final state**, so `result["messages"][-1]` is the answer and `result["messages"]` is the full transcript.

Streaming — watching the loop run

```

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": question}]},
    stream_mode="values",
):
    chunk["messages"][-1].pretty_print()

```

`stream_mode="values"` yields the state after each step, so you watch the transcript grow message by message: your system prompt, the AIMessage carrying tool_calls, each ToolMessage, then the final answer. It is the same sequence your print statements produced in Task 02 — now emitted by the framework.

Your turn — Exercise 3.1: find your own loop inside it

```

for m in result["messages"]:
    label = type(m).__name__
    shown = str(m.content)[:70] or f"tool_calls={m.tool_calls}"
    print(f"{label:<13} {shown}")

```

THE POINT OF THIS TASK

You will see exactly the structure you built by hand. What the framework added is **engineering, not new ideas**: iteration caps, retries on transient errors, parallel execution when the model requests several tools at once, streaming, and consistent error surfaces.

What it did not add is any hidden intelligence. The loop is the loop.

CHECKPOINT 3 — SAVE THIS OUTPUT

Capture the streamed run, and state which parts of the message dump you had constructed manually in Task 02.

TASK 04 • 20 MINUTES • LO2, LO3

Your Own Agent — And Breaking It On Purpose**Part A — Build it (12 minutes)**

Requirements:

- at least three tools of your own, each with real type hints and a docstring written for the model;
- a system prompt that sets both a persona and a tool-use policy;
- a test question that requires chaining two of your tools.

Suggested tools if you are stuck for ideas — pick any three, or invent better ones:

```
unit_convert(value: float, from_unit: str, to_unit: str) -> str
split_bill(total: float, people: int, tip_pct: float) -> str
days_between(start_iso: str, end_iso: str) -> str
bus_fare(route: str) -> str
gpa_points(grade: str) -> str
```

Part B — Experiment 1: sabotage the documentation

Redefine one of your tools with the docstring `"""Does stuff."""` and ask a question that needs it. Run it two or three times.

What you should observe	Selection becomes unreliable — sometimes the right tool, sometimes an apology, sometimes a guessed answer.
Why	Tool choice is a language task performed over your descriptions. Degrade the description and you degrade the choice.
The lesson	This is the cheapest agent bug to create and the cheapest to fix. Write docstrings for the model, not for yourself.

Part C — Experiment 2: poison a result

```
@tool
def flaky_lookup(city: str) -> str:
    """Look up the population of a city."""
    raise RuntimeError("service unavailable")
```

What you should observe: `create_agent` catches the exception and feeds it back to the model as a tool result, so the model apologises, retries or explains — and the run survives. A naive Task 02 loop without the `try/except` would have crashed with a traceback.

STATE THE LESSON PRECISELY

An agent's error handling is not about preventing errors. It is about converting them into observations that the model can act on.

CHECKPOINT 4 — SAVE THIS OUTPUT

Demonstrate your assistant chaining two of your own tools, plus one sentence per experiment on what changed.

Then write down: *what would you add to your Task 02 loop to survive a failing tool?*

TASK 05 • 15 MINUTES • LO2, LO3

Shipping the Agent Behind an API

A notebook is a laboratory. Nothing you have built so far can be called by a web page, a scheduled job or another service — and nobody but you can see what the agent did. The file `api/main.py` fixes both: it exposes the agent over HTTP and returns the trace as data.

Endpoint	Method	Purpose
/health	GET	Liveness and configuration. Makes no model call, so it costs no quota.
/tools	GET	The exact schemas advertised to the model.
/agent/run	POST	Your hand-rolled loop. Returns the answer and the step-by-step trace.
/agent/framework	POST	The same task via <code>create_agent</code> , for comparison.

The request and response models

```
class AskRequest(BaseModel):
    question: str
    max_iterations: int = Field(5, ge=1, le=10)

class Step(BaseModel):
    step: int
    tool: str
    args: dict[str, Any]
    result: str

class AskResponse(BaseModel):
    answer: str
    steps: list[Step]
    stop_reason: Literal["answered", "iteration_cap"]
    messages: int
    total_tokens: int
    seconds: float
```

What those classes buy you, for free:

- **Validation.** A client sending `max_iterations: 99` gets 422 Unprocessable Entity before a single token is spent. The cap is now enforced at the edge as well as inside the loop.
- **Documentation.** FastAPI generates OpenAPI from these classes, so `/docs` becomes a working console with example values.
- **A contract.** `stop_reason` distinguishes “the agent answered” from “the agent was cut off” — the difference between a result and an incident, and something a caller can branch on.

Counting tokens correctly

```
total_tokens += (response.usage_metadata or {}).get(
    "total_tokens", 0
)
```

Token counts live on `usage_metadata`, not `response_metadata`.

IF /agent/run REPORTS total_tokens: 0

Open `api/main.py` and check the line above. Reading token counts from `response_metadata["token_usage"]` silently returns 0 for Gemini, because that key does not exist there — the counts live on `usage_metadata`. Wrapping it in `(... or {})` matters too: usage is not guaranteed on every response, and an observability feature must never be able to break the request it is observing.

Running the service

```
uvicorn api.main:app --reload --port 8000
```

From the lab5 folder, with the venv active.

- `api.main` is the module path (`api/main.py`); `app` is the `FastAPI()` object inside it.
- `--reload` restarts on save — that is how you do Exercise 5.1 without restarting anything else.
- Open <http://127.0.0.1:8000/docs> for a full interactive console generated from the Pydantic models.

The notebook can also start the service in a background thread — the cell binds to port 0 so the operating system hands it any free port, which stops twenty students on one machine colliding.

Your turn — Exercise 5.1

1. Add one of your Task 04 tools to the `TOOLS` list in `api/main.py`.
2. Confirm it appears in `GET /tools`.
3. Ask a question that needs it, and read the returned trace.
4. Then answer, in a comment: the iteration cap is enforced in `AskRequest` (`le=10`) and again inside the loop. **Why must that limit live in the API code rather than in the system prompt?**

CHECKPOINT 5 — SAVE THIS OUTPUT

Show `/agent/run` returning a two-step trace, and your extended `/tools` output.

QUICK TEST • 10 MINUTES**Knowledge Check**

Answer all questions in your Word document. Write the question number and your answer. Questions 1–5 are multiple choice; 6 and 7 are short answer; 8 is scenario-based.

Multiple choice (choose one)

Q1. The chat API has no memory of your previous call because:

- (a) the model deliberately forgets for privacy
- (b) the endpoint is a pure function of the messages you send
- (c) the free tier disables memory
- (d) the temperature is set to zero

Q2. When an `AIMessage` comes back with `tool_calls` populated and `content` empty, it means:

- (a) the request failed
- (b) the model has executed a function and is returning the result
- (c) the model is requesting that YOUR code execute a function
- (d) the tool schema was invalid

Q3. The tool description that the model uses when deciding is derived from:

- (a) the function name only
- (b) the function's docstring
- (c) a separate JSON file you write
- (d) the type hints only

Q4. The correct place to enforce an agent's iteration cap is:

- (a) in the system prompt
- (b) in the model's temperature setting
- (c) in your code, as an integer bound on the loop
- (d) in the tool docstrings

Q5. A 429 / `ResourceExhausted` error from the Gemini free tier most often means:

- (a) your API key is invalid
- (b) the model ID does not exist
- (c) you exceeded requests per minute — wait and retry
- (d) your daily quota is permanently gone

Short answer (two or three sentences each)

Q6. Name three agent failure modes and state the code-level guardrail for each.

Q7. In your Task 02 loop, the token count grew faster than the number of steps. Explain why, in terms of what is sent on each iteration.

Scenario-based

Q8. A teammate proposes controlling your agent's tool use entirely through the system prompt: "Never call a tool more than five times, and never call `delete_order`." Give two distinct reasons why this is not an adequate safety design, and state what you would implement instead.

PUT YOUR QUICK-TEST ANSWERS IN THE SAME WORD DOCUMENT

Place them in a clearly labelled "Quick Test" section after your Task 01–05 evidence.

Submission Instructions

Submit TWO files to CourseWeb before the end of the 2-hour session.

File	What it must contain
ITXXXXXXXX_Lab05.docx	Your name, IT number, lab number; the five checkpoint outputs (screenshots or pasted text); your answers to Exercises 1.1, 2.4, 5.1; and your Quick Test answers Q1–Q8.
ITXXXXXXXX_Lab05_notebook.ipynb	Your completed lab.ipynb with Task 02 and Task 04 written by you, all cells run, and the outputs saved.

File naming and upload

1. Rename both files using your IT number in this exact format: **ITXXXXXXXX_Lab05.docx** and **ITXXXXXXXX_Lab05_notebook.ipynb**
2. Example: **IT22123456_Lab05.docx**
3. Upload both to CourseWeb before the end of the 2-hour lab session.
4. **Late submissions will not be accepted** unless prior approval is given by the lecturer.

DO NOT SUBMIT YOUR .env FILE OR YOUR API KEY

Before you save the notebook, check that no cell output contains your key. If your key has ever appeared in a screenshot or a shared file, revoke it at <https://aistudio.google.com/apikey> and create a new one. Submitting a live credential is a security incident, not a formatting mistake.

Evaluation Criteria (Marking Guide — 20 Marks)

This is how your submission will be marked. Note that Task 02 alone carries 7 of the 20 marks.

Component	Marks	What earns full marks
Task 00 + Task 01 — environment and raw chat	3	Environment green; amnesia demo and the fix both shown; persona variant present and the persona correctly explained as a re-sent message.
Task 02 — the hand-rolled loop	7	Loop written by the student and working; trace shows two chained tool calls; all three guardrails present (unknown-tool check, try/except, iteration cap); instrumentation reports messages and tokens; the token-growth question answered correctly.
Task 03 — create_agent	2	Streamed run captured; student correctly identifies which parts of the transcript match their own Task 02 structure.
Task 04 — own agent and break-it experiments	3	Three own tools with model-facing docstrings; two tools chained; both experiments run with a correct one-sentence lesson each.
Task 05 — the API endpoint	3	/agent/run returns a multi-step trace; a student tool added and visible in /tools; the iteration-cap question answered correctly.
Quick Test Q1–Q8	2	Q1–Q5 correct; Q6–Q8 show correct understanding of guardrails, transcript growth and prompt-versus-code enforcement.
Total	20	

Troubleshooting — During the Lab

Exact error text or symptom	Cause and fix
PermissionDenied / “API key not valid”	Key wrong, expired, or pasted with a trailing space. Re-copy it from https://aistudio.google.com/apikey and restart the kernel.
NotFound / “model not found”	Model ID typo. Use a plain ID such as gemini-2.5-flash-lite — no prefix, no URL.
ResourceExhausted (429)	Free-tier rate limit, almost always per minute. Wait about 60 seconds and rerun. If it repeats, set <code>CHAT_MODEL=gemini-2.5-flash-lite</code> and restart the kernel.
DeadlineExceeded, or a connection error	Network, proxy or VPN blocking <code>generativelanguage.googleapis.com</code> . Try without the VPN.
<code>tool_calls</code> is always empty	You called <code>llm.invoke</code> instead of <code>llm_with_tools.invoke</code> — or the question genuinely needs no tool.
KeyError on a tool name	The model hallucinated a tool. Print the name; use <code>.get()</code> and return an error string instead of indexing.
AttributeError: 'list' object has no attribute 'strip'	The model returned content blocks rather than a plain string. Use the <code>text_of()</code> helper from Task 01.
The loop never ends	No iteration cap, or a tool always errors and the model keeps retrying. Both are covered in Task 02.
BadRequestError about a tool schema	A tool is missing type hints or a docstring, so no schema can be derived from it.
ModuleNotFoundError on any lab package	Wrong Jupyter kernel, or packages installed outside the venv. Run the notebook’s first cell.
422 Unprocessable Entity from the API	Your request violated AskRequest. Read the response body — it names the offending field.
The Part 5 cell hangs	An old server thread is still bound to the port. Restart the kernel and rerun the cell.
<code>/agent/run</code> returns <code>total_tokens: 0</code>	<code>api/main.py</code> is reading <code>response_metadata</code> instead of <code>usage_metadata</code> . See the callout in Task 05.

Glossary

Term	Meaning as used in this lab
Agent	A language model in a loop with tools and state.
Tool	A Python function exposed to the model via a name, a description and an argument schema.
Tool call	The model’s structured request that your runtime execute a tool. It does not execute anything itself.
Transcript	The list of messages re-sent on every model call — the only “memory” that exists.
ReAct	Reason, act, observe, repeat: the pattern your Task 02 loop implements.
Trace / trajectory	The ordered record of what the agent did. Returned by <code>/agent/run</code> ; evaluated formally in Lecture 07.
Iteration cap	A hard stop in code that bounds the loop. The prompt cannot be trusted with this.
Temperature	The randomness of token sampling. Use 0 for debugging, higher for variety.
Rate limit (429)	Google refusing a request because you exceeded requests per minute or per day. Usually means “wait”, not “stop”.

Term	Meaning as used in this lab
Usage metadata	LangChain's provider-neutral token counts on a response: input_tokens, output_tokens, total_tokens.

Lab Connection and References

Lab connection: this lab implements the ReAct loop introduced in Lecture 05 and produces the trace that Lecture 07 evaluates. The FastAPI service you build here is the same integration shape you need for the agentic AI feature of the Main Assignment — your React frontend calls the agent exactly as it calls any other REST endpoint.

- **Lecture 05 slide deck** — SE3090: Agentic Software Development 1 (primary reference).
- Biswas, A. and Talukdar, W. (2025) Building Agentic AI Systems. Birmingham: Packt Publishing.
- Google AI for Developers — Gemini API rate limits: <https://ai.google.dev/gemini-api/docs/rate-limits>
- Your live project limits: <https://aistudio.google.com/rate-limit>
- LangChain documentation — agents and tools: <https://docs.langchain.com>
- FastAPI documentation: <https://fastapi.tiangolo.com>